

Lesson 4 — Input And Movement

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

Blocking vs non-blocking

```
std::cin >> command;    // STOPS and waits
int key = read_key();    // returns 0 and
                        carries on
```

A real-time game **never waits**.

Raw mode (Unix)

```
void start_raw_mode() {
    tcgetattr(STDIN_FILENO, &original);
    termios raw = original;
    raw.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &raw);
    fcntl(STDIN_FILENO, F_SETFL,
        fcntl(STDIN_FILENO, F_GETFL, 0) |
        O_NONBLOCK);
}

void stop_raw_mode() {          // NOT OPTIONAL
    tcsetattr(STDIN_FILENO, TCSANOW,
        &original);
}
```

- `ICANON` — line buffering (waits for Enter)
- `ECHO` — printing what you type
- `O_NONBLOCK` — `read()` returns immediately

If the terminal breaks: type **reset** and press **Enter**. You will not see yourself typing.

Why `#ifdef`

Reading a key without waiting is **not standard C++**. The language says nothing about terminals.

```
#ifdef _WIN32
    #include <conio.h>          // _kbhit(),
    _getch()
#else
    #include <termios.h>
#endif
```

Arrow keys are THREE characters

27, then 91, then 65 / 66 / 67 / 68
(up/down/right/left).

Check first, then move

```
int nx = x + dx, ny = y + dy;
char target = maze_at(nx, ny);
if (target == '#') return;    // do not move
x = nx; y = ny;
```

Validate, then act. Not: move, check, move back.

Direction pairs

```
'w' -> (0, -1)    // up is MINUS: row 0 is
the top
's' -> (0, +1)
'a' -> (-1, 0)
'd' -> (+1, 0)
```

Section A — Recall

A1. What is the difference between blocking and non-blocking input? Which does a real-time game need? [2 marks]

A2. Name the two terminal behaviours raw mode switches off, and say why a game does not want each. [3 marks]

A3. Why must you call `stop_raw_mode()`? What does the user see if you do not, and what is the cure? [3 marks]

A4. Why does this code need `#ifdef` when nothing else in the course has? [2 marks]

A5. How many characters does pressing the up arrow send, and what are they? [2 marks]

Section B — Predict the output

B1. The player is at `(5, 3)` and presses `w`. What square does the game check, and why is it *minus one*? [3 marks]

B2. What happens if `O_NONBLOCK` is left out of `start_raw_mode()`? Describe exactly what the player sees. [4 marks]

B3. The player presses the up arrow. What does this print, and why three lines? [3 marks]

```
int key = read_raw_key();
if (key != 0) std::cout << key << "\n";
```

(Called once per frame, three frames in a row.)

B4. A player walks into a wall. Describe what happens with each version, and say which is better and why. [4 marks]

```
// version A
char target = maze_at(x + dx, y + dy);
if (target == '#') return;
x += dx; y += dy;

// version B
x += dx; y += dy;
if (maze_at(x, y) == '#') { x -= dx; y -= dy; }
```

Section C — Find and fix the bug

C1. The game freezes until a key is pressed, then does exactly one frame and freezes again. What is missing? [3 marks]

C2. The player walks straight through walls. The maze data is correct. What is wrong? [4 marks]

```
void try_move(int dx, int dy) {
    char here = maze_at(player_x, player_y);
    if (here == '#') return;
    player_x += dx;
    player_y += dy;
}
```

C3. The game works perfectly, but after quitting, the student's terminal no longer shows what they type. What did they forget, and what should they type to fix the terminal? [3 marks]

Section D — Build it

- ☐ **Checkpoint 1.** Get `read_key()` working. Print the code of each key pressed, with a heartbeat message so you can see the loop running while you press nothing.
- ☐ **Checkpoint 2.** Add `start_raw_mode()` and `stop_raw_mode()`. **Test that quitting leaves your terminal working.**
- ☐ **Checkpoint 3.** Draw a maze from an array of strings, with a player `@`. Move the player with WASD. Walking through walls is fine for now.

- ☐ **Checkpoint 4.** Add the wall check — **check the destination, then move.** Walk round the whole maze and check you cannot escape.
- ☐ **Checkpoint 5.** Add collectable keys that disappear when you walk onto them, and a counter.
- ☐ **Checkpoint 6.** Add a locked exit that only works once all the keys are collected, and something that animates (a flickering torch, a countdown) so the screen is alive while you stand still.

WHEN YOUR TERMINAL BREAKS

It will. Type `reset` and press Enter. You will not see yourself typing, and it will work anyway. Write that on a sticky note now.</p>

Also: **run from a real terminal**, not an IDE's built-in console. Many of them do not pass individual keystrokes through, and your perfectly correct program will appear dead.</p>

Section E — Design challenge

NO RIGHT ANSWER

E1. `read_key()` reads **one** key per frame. What happens if the player presses three keys very quickly within one frame? Is it a problem? **How would you find out?**

E2. The maze is written in the code. Design how you would load it from a **file** so levels could be made without recompiling. What could go wrong with a file a player has edited, and what should your program do about each case?

E3. Your program changes a **system-wide** setting and has to put it back. What else might a game change that it must restore? What happens if the program **crashes** before the restore runs?

E4. Non-blocking input needed different code for Windows and Unix, because standard C++ says nothing about terminals. **Why would the language leave out something so obviously useful?** What would it cost to put it in?

Stretch goals

1. **Push blocks** you can shove into empty space. You now have Sokoban — and some real design decisions about what happens when a crate hits another crate.
2. Load the maze from a file.
3. A patrolling guard on its own timer. Notice that **none of your input code changes**.
4. A field of view, so you only see squares near you.
5. **Hold-to-move**, with a short delay before the repeat starts. Genuinely awkward in a terminal — find out why.